



RISC-V ELF DECODER



Submitted by:
Sarah Abdelatty Ibrahim

Table of Contents

.1	Project Overview	3
1.1	What the program does, in one pipeline	3
1.2	Key features	3
2.	Full Architecture	4
2.1	Layer responsibilities	5
2.2	Data contracts between layers.....	5
3.	Stage 1 - The Driver (main.cpp)	6
3.1	Output line format.....	6
4.	Stage 2 - ELF Parsing (elf_types.hpp, elf_reader.hpp).....	7
4.1	What is read from the file	7
4.2	Loading and validation flow	8
5.	Stage 3 - Instruction Decoding (riscv_disasm.hpp)	9
5.1	Entry point: decode_one().....	9
5.2	32-bit instruction formats.....	9
5.3	decode32(): dispatch on opcode.....	11
5.4	decode16(): compressed instructions	11
6.	Stage 4 - Vector Extension (riscv_vector.hpp)	13
7.	9. Build, Test and Verification.....	14
7.1	Building.....	14
7.2	Test layers	14
7.3	Test inputs	15
7.4	Actual output of the decoder	15

1. Project Overview

riscv_decoder is a command-line disassembler. You give it the path of a RISC-V ELF executable; it opens the file, parses the ELF structures by hand (no libelf, no binutils), finds every executable section, and decodes the machine code inside them into human-readable assembly. The output format deliberately imitates **objdump -d**, so it can be compared line by line against the real tool.

1.1 What the program does, in one pipeline

```

ELF file  -> parse headers -> find code sections -> decode each
on disk   (Ehdr, Shdr)   (SHF_EXECINSTR)    instruction
                                     |
                                     |-----> print
                                     (<main>: ...)  objdump-style text

```

1.2 Key features

- **Both ELF classes:** ELF32 (RV32) and ELF64 (RV64) share one templated parser.
- **Mixed instruction widths:** 32-bit instructions and 16-bit compressed (RVC) instructions are told apart by the two lowest bits.
- **Wide ISA coverage:** RV32I/RV64I, M, A, C (plus Zcb **c.lbu**), Zba, Zbb, Zbs, Zicond, Zihintpause, Zicsr, FENCE/FENCE.I, privileged returns (**mret**, **sret**, **wfi**) and the full-featured RVV 1.0 vector extension.
- **objdump-style pseudo-instructions:** **li**, **mv**, **ret**, **j**, **jr**, **nop**, **seqz**, **snez**, **csrr**, **csrw** and others are printed the way a real assembler listing shows them.
- **Symbol labels:** addresses that match a symbol get a **<name>:** heading, just like objdump.
- **Defensive parsing:** every read is range-checked, so a truncated or corrupt ELF produces a clear error instead of a crash.
- **Header-only core:** the whole decoder lives in four headers, which makes it trivial to compile into unit tests.

2. Full Architecture

The system has three layers between the input file and the printed output. Data flows downward as bytes, and results flow back up as structured values.

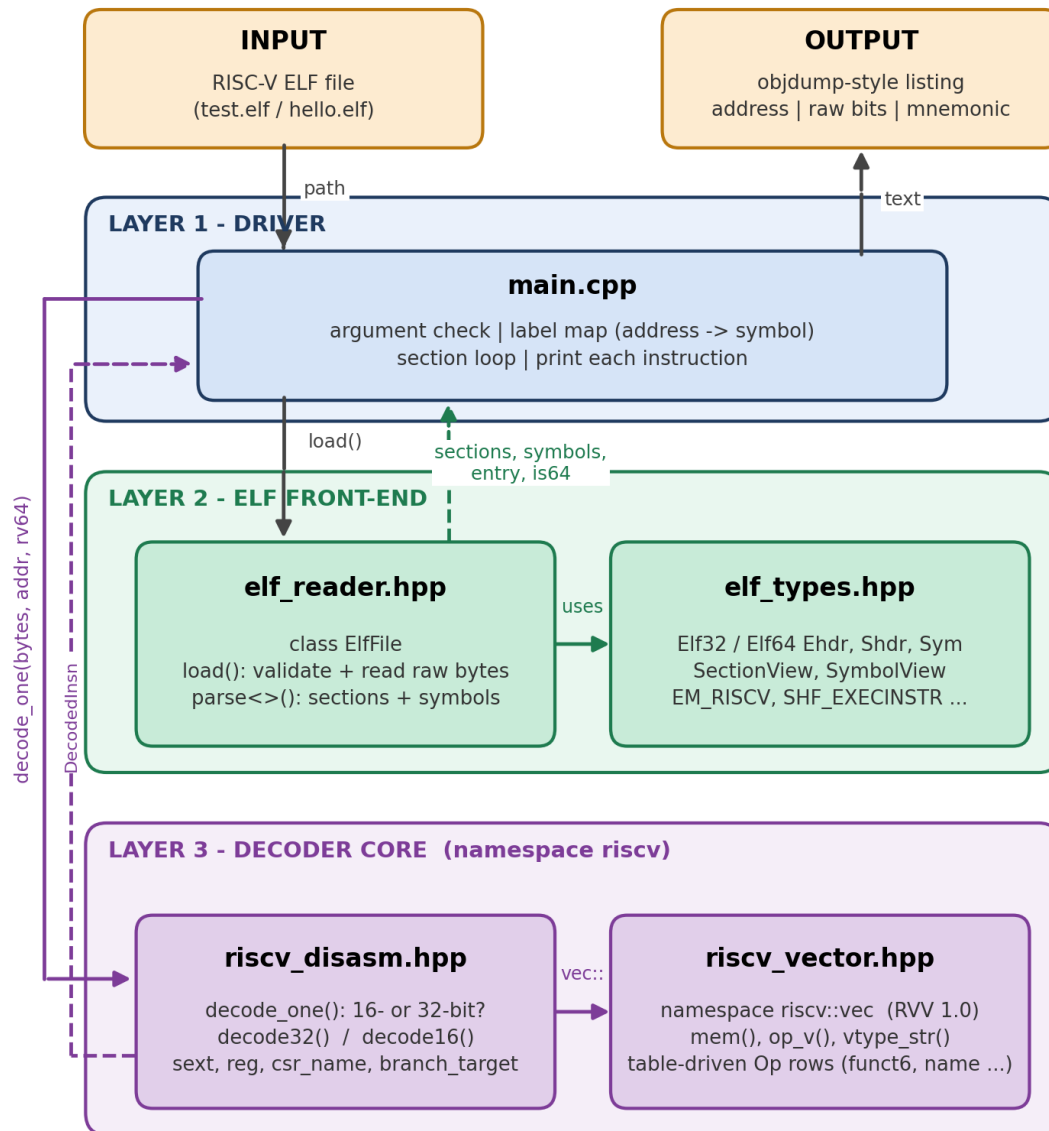


Figure 1 - Full system architecture and data flow

2.1 Layer responsibilities

Layer	Files	Responsibility
1. Driver	<code>main.cpp</code>	Validates arguments, calls the ELF reader, builds the address-to-symbol map, walks every code section, calls the decoder, and formats each output line. Owns all printing.
2. ELF front-end	<code>elf_types.hpp</code> <code>elf_reader.hpp</code>	Turns a file on disk into a clean, class-independent model: entry point, list of executable sections (SectionView) and list of symbols (SymbolView). Owns all file-format knowledge and bounds checking.
3. Decoder core	<code>riscv_disasm.hpp</code> <code>riscv_vector.hpp</code>	Turns (bytes, address, rv64 flag) into a DecodedInsn: raw bits, size, mnemonic text. Owns all ISA knowledge. Has no I/O and no ELF dependency.

2.2 Data contracts between layers

Contract	Fields and meaning
SectionView	<code>name</code> , <code>addr</code> (virtual address), <code>offset</code> (position in the file), <code>size</code> , <code>execinstr</code> . Tells main.cpp where the code bytes are and what address they should be printed at.
SymbolView	<code>name</code> , <code>value</code> (address), <code>shndx</code> . Used to print <code><name>:</code> labels.
ElfFile	<code>is64</code> , <code>little_endian</code> , <code>entry</code> , <code>exec_sections</code> , <code>symbols</code> , <code>raw</code> (entire file in memory).
<code>riscv::DecodedInsn</code>	<code>raw</code> (16 or 32 bits), <code>size</code> (2 or 4, or 1/remaining for truncated tails), <code>text</code> (the mnemonic and operands).
<code>decode_one(...)</code>	Signature: <code>DecodedInsn decode_one(const unsigned char* data, size_t avail, uint64_t addr, bool rv64)</code> . This single function is the whole public API of the decoder.

3. Stage 1 - The Driver (main.cpp)

main.cpp is intentionally thin. It coordinates the other layers and produces the text. It contains no ELF parsing and no instruction decoding.

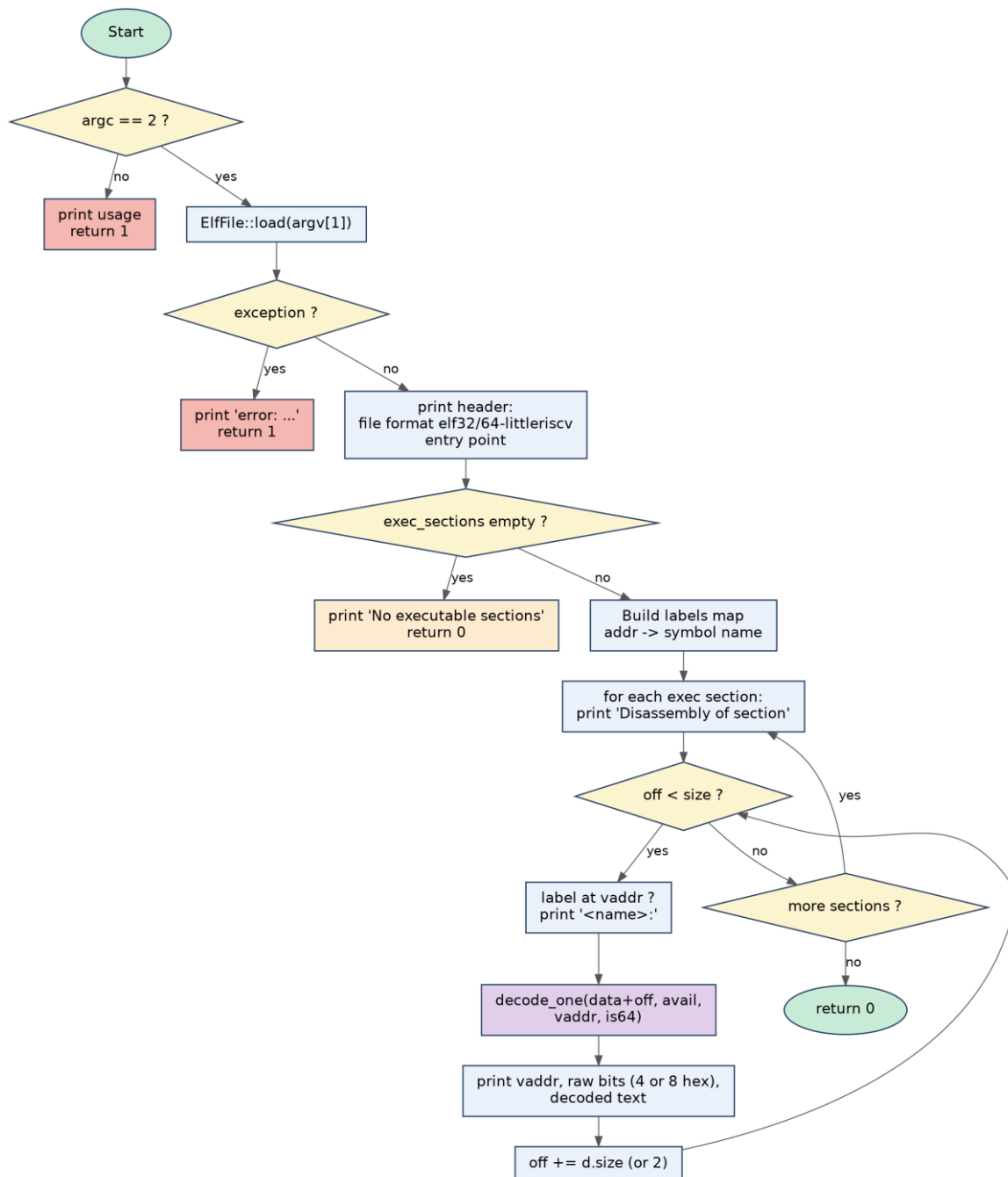


Figure 2 - Flowchart of main()

3.1 Output line format

Each instruction line has three tab-separated columns. The address is zero-padded to 12 hex digits for RV64 and 8 for RV32. The raw column is 4 hex digits for a compressed instruction and 8 for a normal one, so you can see the instruction width at a glance.

000000010000:	00000513	li a0,0	<- 32-bit (8 hex digits)
00000001000c:	4501	li a0,0	<- 16-bit (4 hex digits)
address	raw	text	

4. Stage 2 - ELF Parsing (elf_types.hpp, elf_reader.hpp)

The ELF layer answers three questions: **is this a valid RISC-V ELF**, **where is the code**, and **what are the names of things**? It reads only the parts of the format it needs.

4.1 What is read from the file

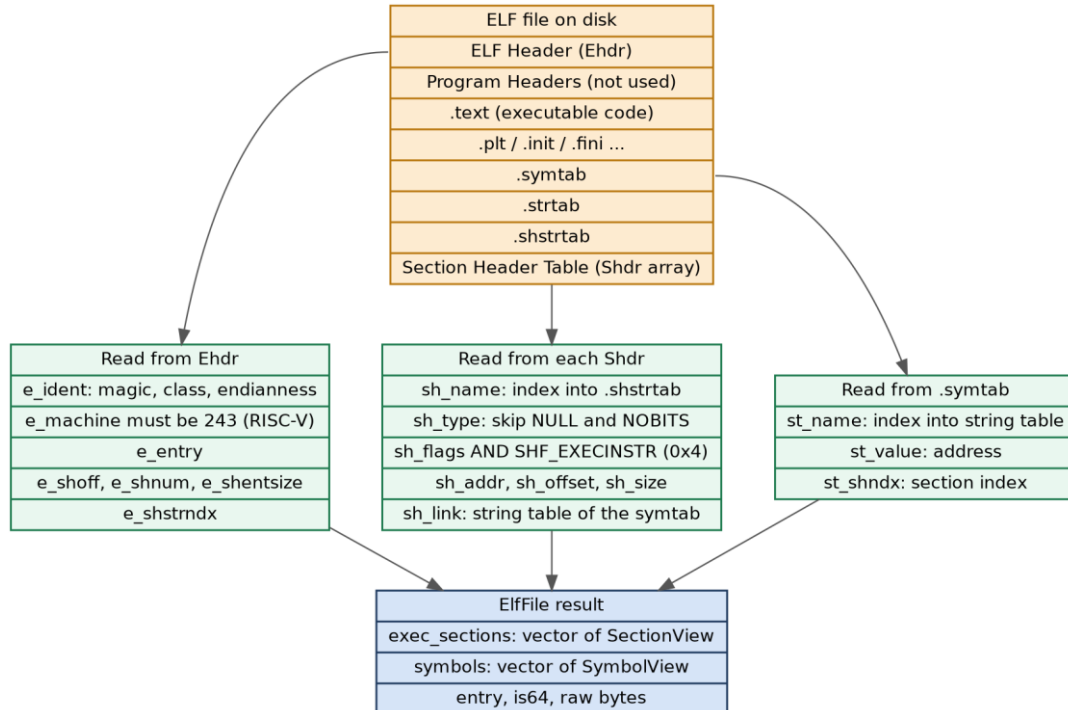


Figure 3 - ELF regions consulted by the parser

4.2 Loading and validation flow

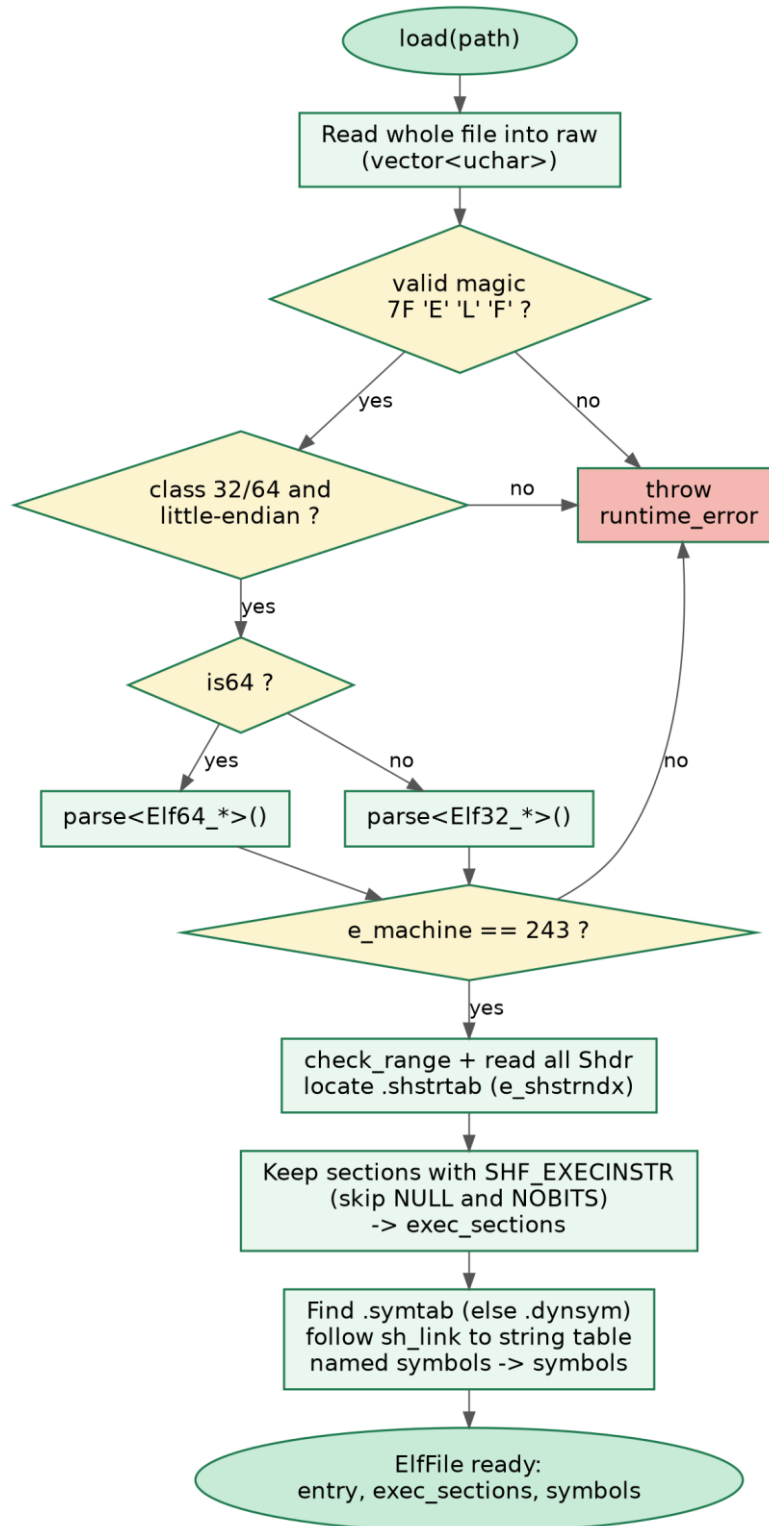


Figure 4 - ElfFile::load() and parse<>() flowchart

5. Stage 3 - Instruction Decoding (riscv_disasm.hpp)

This is the heart of the project. It converts raw bytes into text. The design has three levels: **decode_one** finds out how long the instruction is, then hands it to **decode32** or **decode16**, which switch on the opcode fields.

5.1 Entry point: decode_one()

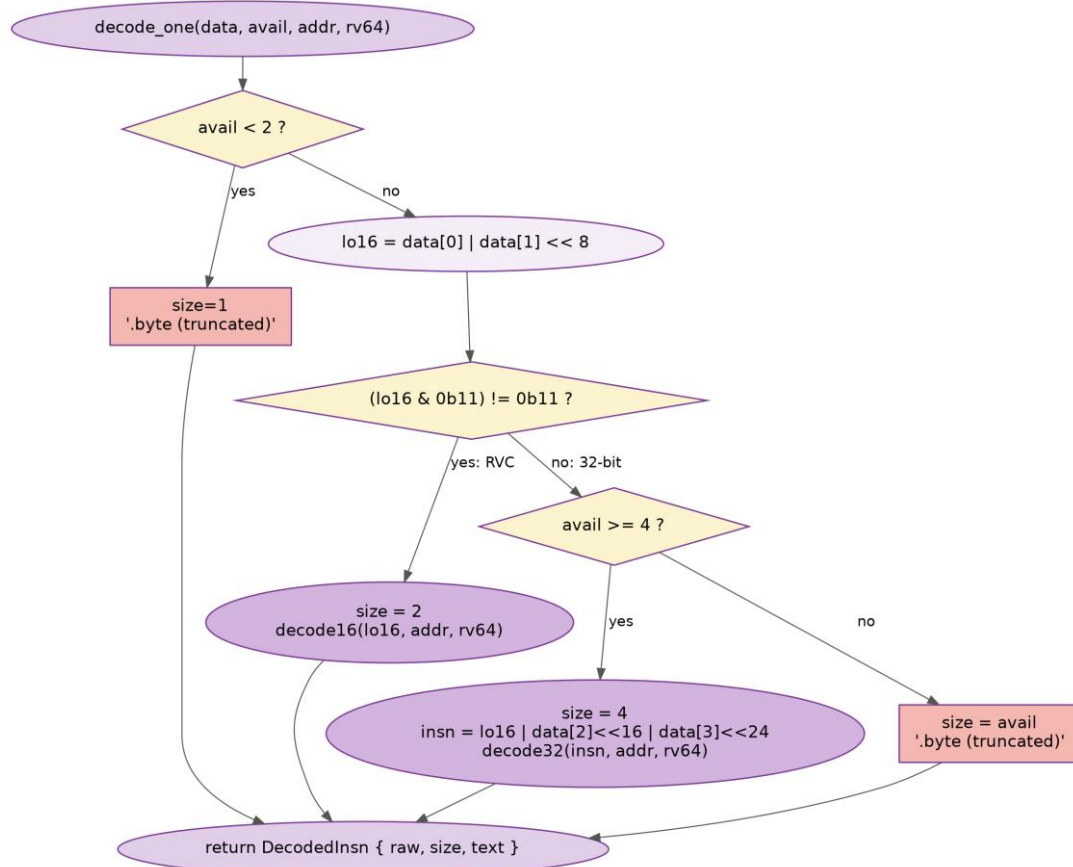


Figure 5 - `decode_one()`: choosing 16-bit or 32-bit decoding

RISC-V encodes instruction length in the lowest bits of the first 16-bit parcel. If the two lowest bits are anything other than **11**, the instruction is a 16-bit compressed one. If they are **11**, it is a 32-bit instruction. Bytes are combined little-endian. The function also protects against reading past the end of the section (`avail < 2`, or `avail < 4` for a 32-bit opcode).

5.2 32-bit instruction formats

Every 32-bit RISC-V instruction has its opcode in bits 6:0. The other fields sit at fixed positions, which is why `decode32` can extract `rd`, `funct3`, `rs1`, `rs2` and `funct7` once, at the top, before it knows the format. Only the immediate is assembled differently per format.

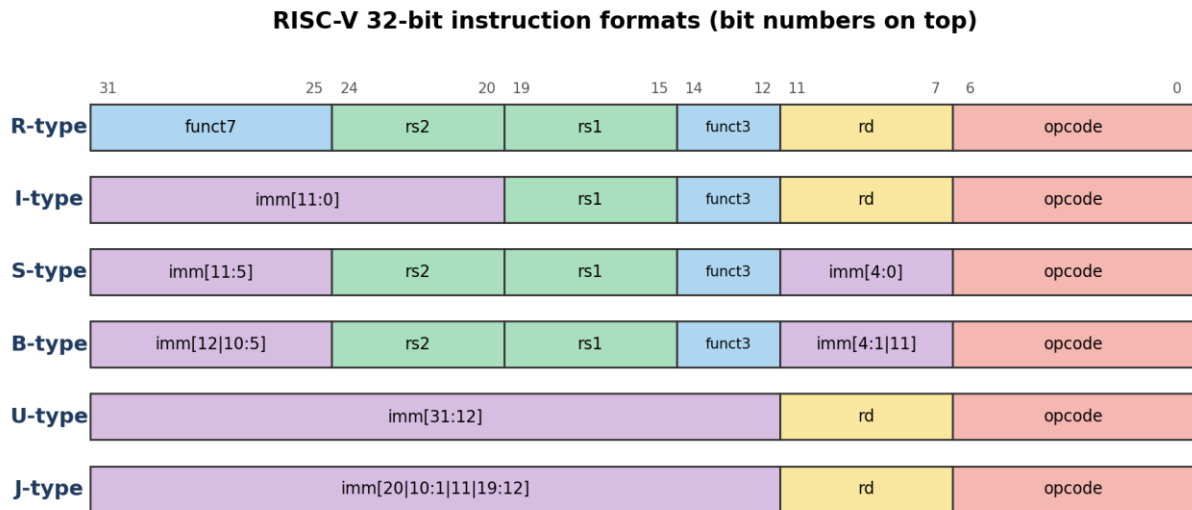


Figure 6 - The six base instruction formats

Format	Immediate in the code	Used by
R	none	add, sub, and, or, xor, sll, srl, sra, slt, M-extension, Zba/Zbb/Zbs
I	$\text{imm_i} = \text{sext}(\text{insn} \gg 20, 12)$	addi, loads, jalr, shifts, CSR ops
S	$\text{imm_s} = \text{sext}((\text{insn} \gg 25) \ll 5 \mid (\text{insn} \gg 7) \& 0x1f, 12)$	sb, sh, sw, sd
B	imm_b (13 bits, bit 0 always 0)	beq, bne, blt, bge, bltu, bgeu
U	$\text{insn} \gg 12$	lui, auipc
J	imm_j (21 bits, bit 0 always 0)	jal

5.3 decode32(): dispatch on opcode

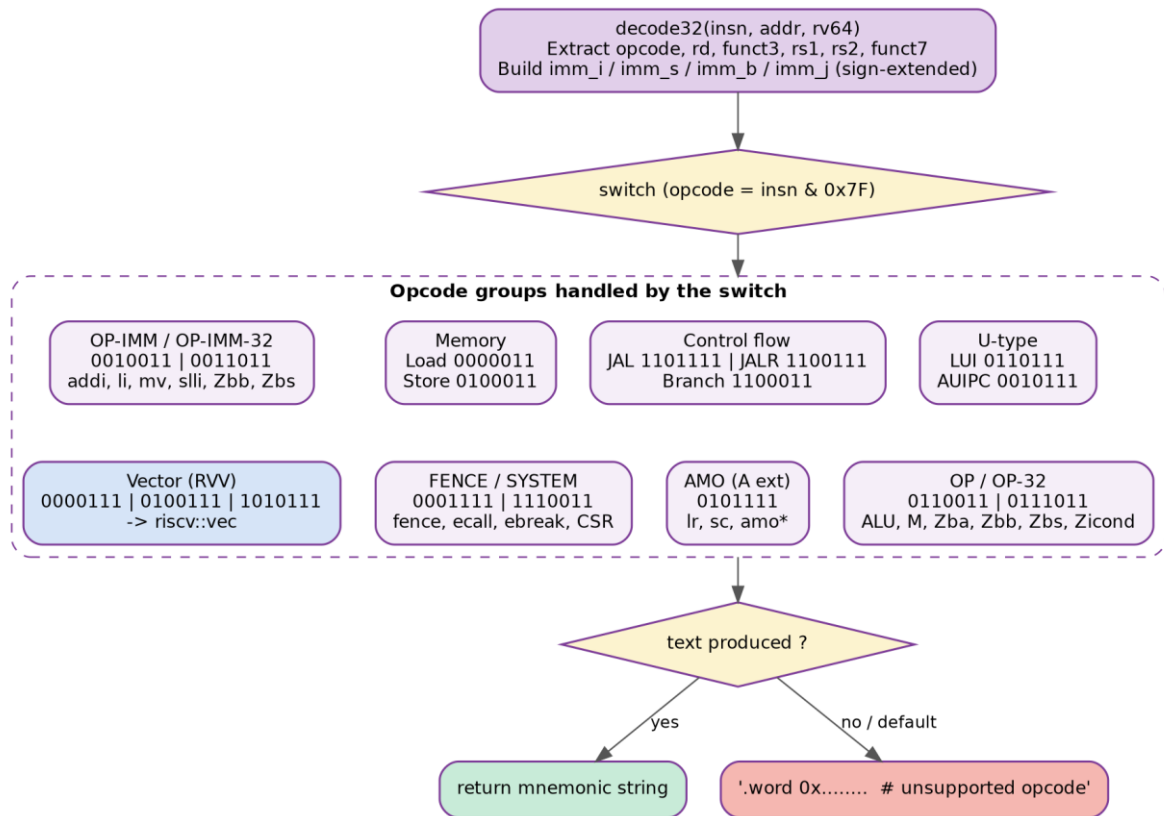


Figure 7 - `decode32()` control flow

5.4 decode16(): compressed instructions

Compressed instructions have three quadrants selected by bits 1:0 (00, 01, 10) and a 3-bit funct3 in bits 15:13. Their immediates are scattered across the word in a hardware-friendly but human-unfriendly order, so each case re-assembles the bits by hand and then sign-extends. Most compressed forms can only name registers x8-x15, decoded with `creg()`.

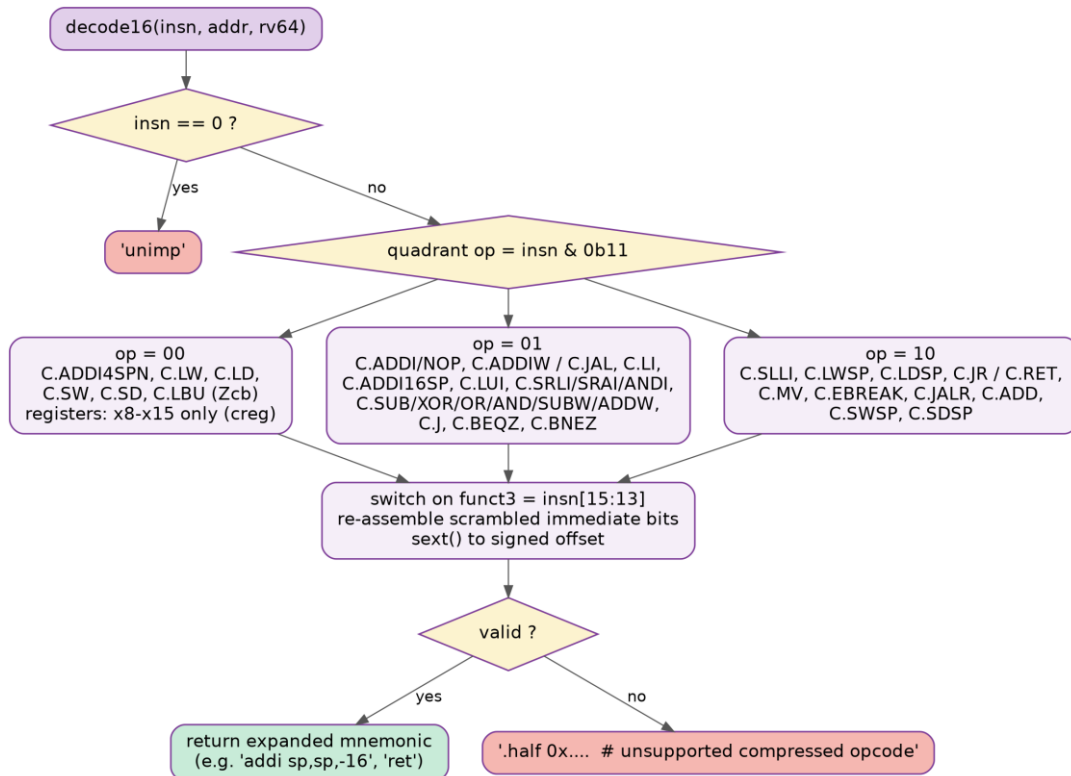


Figure 8 - decode16() control flow

Compressed instructions are printed in their **expanded** form (for example `c.addi sp,-16` shows as `addi sp,sp,-16`), which is what `objdump` shows by default. Encodings that are illegal (all-zero word, zero immediate where a non-zero one is required) print as `unimp`.

6. Stage 4 - Vector Extension (riscv_vector.hpp)

The vector unit is a large, regular instruction set (RVV 1.0). Rather than writing hundreds of switch cases, `riscv_vector.hpp` is **table-driven**: each vector operation is one row in a small array, and one generic function turns a row into text.

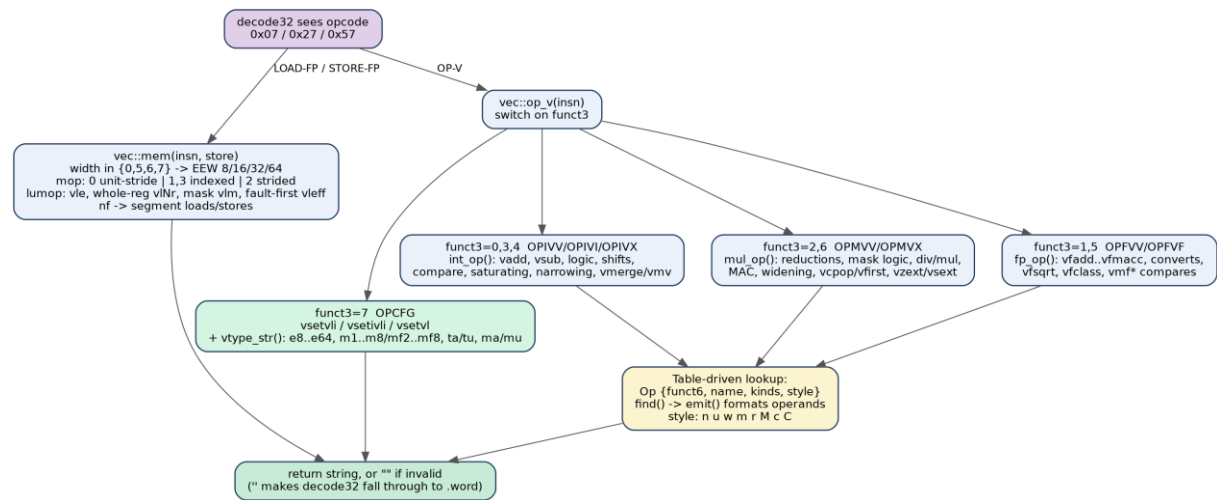


Figure 9 - Vector decoding path

7. 9. Build, Test and Verification

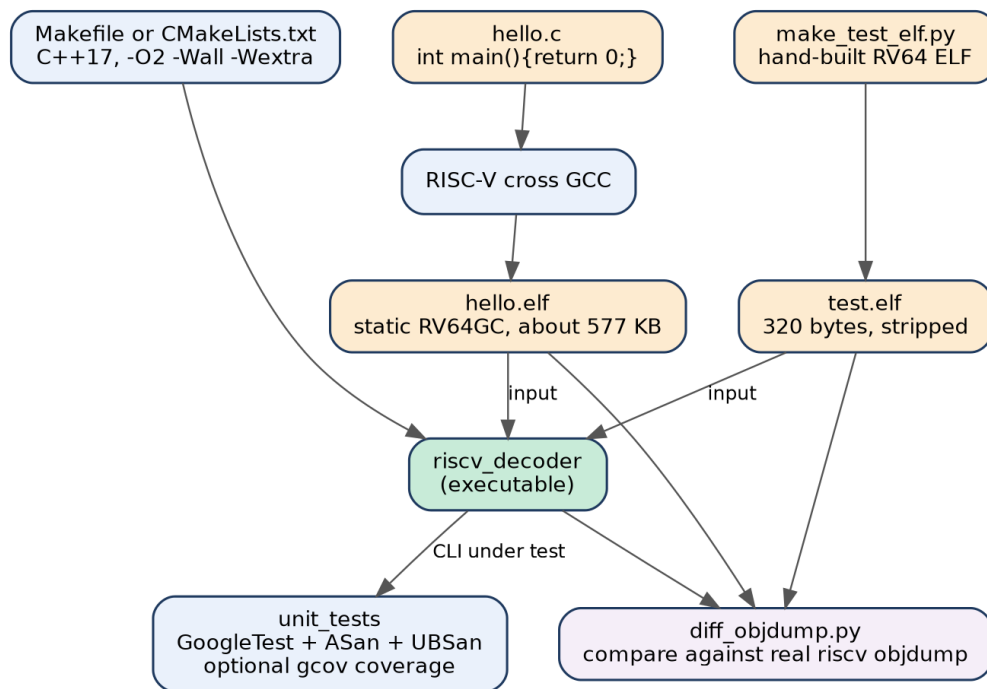


Figure 10 - Build and verification workflow

7.1 Building

```

# Makefile
make          # g++ -std=c++17 -O2 -Wall -Wextra -o riscv_decoder main.cpp

# CMake
cmake -S . -B build && cmake --build build

# Run
./riscv_decoder test.elf
./riscv_decoder hello.elf | less
  
```

7.2 Test layers

Layer	What it verifies
GoogleTest unit tests	Seven test files: utils, decode32, decode16, decode_one, elf_reader, cli, vector. Built with AddressSanitizer + UBSan by default (ENABLE_SANITIZERS=ON), so memory or undefined-behaviour bugs fail the tests. The CLI test runs the real binary (path injected through RISCV_DECODER_BIN).
Differential test vs objdump	tests/tools/diff_objdump.py runs both riscv_decoder and a real riscv64-* -objdump on test.elf and hello.elf and compares. Registered only if an objdump and Python 3 are found. This is the strongest correctness check because the reference is an independent implementation.
Coverage	ENABLE_COVERAGE=ON (CMake) or make coverage (gcovr) reports line coverage of the headers.

7.3 Test inputs

test.elf is produced by `make_test_elf.py`. The script packs an ELF64 header, a 12-instruction `.text` (mixing 32-bit and compressed opcodes, a branch and a self-jump) and a `.shstrtab`, then writes section headers by hand with `struct.pack`. It is 320 bytes and stripped, so it also exercises the no-symbols path.

hello.elf is `hello.c` (`int main(){return 0;}`) compiled with a RISC-V cross compiler, statically linked. Although the source is one line, the binary contains the whole C library (about 98,900 lines of disassembly, 2,300+ symbols), making it a realistic stress test for symbols, compressed code and unusual instructions.

7.4 Actual output of the decoder

Running `./riscv_decoder test.elf` produced:

```
test.elf:  file format elf64-littleriscv
entry point: 0x10000

Disassembly of section .text:

000000010000: 00000513  li a0,0
000000010004: 00100593  li a1,1
000000010008: 00b58633  add a2,a1,a1
00000001000c: 4501     li a0,0
00000001000e: 0585     addi a1,a1,1
000000010010: 00078793  mv a5,a5
000000010014: 00c70c63  beq a4,a2,0x1002c
000000010018: ff010113  addi sp,sp,-16
00000001001c: 00a13423  sd a0,8(sp)
000000010020: 8082     ret
000000010022: 00008067  ret
000000010026: 0000006f  j 0x10026
```

And the start of **hello.elf** (symbols are present, so labels appear):

```
Disassembly of section .plt:

10240 <__rela_iplt_end>:
000000010240: 00077e17  auipc t3,0x77
000000010244: dc0e3e03  ld t3,-576(t3)
000000010248: 000e0367  jalr t1,0(t3)
00000001024c: 00000013  nop

Disassembly of section .text:

10250 <_dl_start>:
000000010250: 1141     addi sp,sp,-16
000000010252: e022     sd s0,0(sp)
000000010254: e406     sd ra,8(sp)
```